

CSES String Algorithms

A Problem-Solving Companion to the CSES Problem Set

Maruri Saikiran

August 10, 2026

Contents

1	String Algorithms	1
1.1	The Five Machines	1
1.2	Word Combinations	2
1.3	String Matching	3
1.4	Finding Borders	5
1.5	Finding Periods	6
1.6	Minimal Rotation	8
1.7	Longest Palindrome	10
1.8	All Palindromes	11
1.9	Required Substring	13
1.10	Palindrome Queries	14
1.11	Finding Patterns	16
1.12	Counting Patterns	18
1.13	Pattern Positions	21
1.14	Distinct Substrings	22
1.15	Distinct Subsequences	24
1.16	Repeating Substring	26
1.17	String Functions	28
1.18	Inverse Suffix Array	30
1.19	String Transform	31
1.20	Substring Order I	33
1.21	Substring Order II	35
1.22	Substring Distribution	37

Chapter 1

String Algorithms

String algorithms are a small set of powerful machines – the prefix and Z functions, Manacher’s palindrome radii, polynomial hashing, the suffix array, and the suffix automaton – reused across many problems. The CSES string section is really a tour of these five machines: learn what each computes and the problems become a matter of feeding the right string to the right machine.

1.1 The Five Machines

Almost every problem here is solved by one of a handful of linear or near-linear tools. Recognising which one a problem needs is the whole game.

Which Machine Does the Problem Need?

- **Prefix function / Z-function** – borders, periods, exact pattern matching, “string built from repeated blocks”: Finding Borders, Finding Periods, String Matching, String Functions, Required Substring.
- **Manacher’s algorithm** – everything about palindromic substrings in $O(n)$: Longest Palindrome, All Palindromes.
- **Polynomial hashing** – equality of substrings in $O(1)$ after $O(n)$ preprocessing: Palindrome Queries, Repeating Substring, Finding Patterns.
- **Suffix array + LCP** – ordering and counting substrings, reconstruction: Distinct Substrings, Inverse Suffix Array, Repeating Substring.
- **Suffix automaton** – the Swiss-army knife for substring counting and ranking: Counting Patterns, Pattern Positions, Distinct Substrings, Substring Order I/II, Substring Distribution.
- **Standalone constructions** – Minimal Rotation (Booth’s), Word Combinations (trie + DP), Distinct Subsequences (DP), String Transform (inverse Burrows–Wheeler).

Two Ideas Underlie Most of Them

Borders. A border is a proper prefix that is also a suffix; the prefix function stores the longest border of every prefix, and its “failure chain” enumerates all borders – the basis of matching, periods, and automaton building. **Endpos classes.** Two substrings are equivalent if they occur at exactly the same set of ending positions; the suffix automaton has one state per endpos class, which is why it counts and ranks substrings so cheaply.

Each section names the machine, explains why it applies, gives complete compilable C++, a dry run, and complexity. Where a suffix automaton is used, the same compact construction reappears so every listing stands alone.

1.2 Word Combinations

Problem Statement. Given a string s of length n and a dictionary of k words, count the number of ways to build s as a concatenation of dictionary words (a word may be reused). Output the count modulo $10^9 + 7$.

Pattern Recognition

“Count ways to segment a string into dictionary words” is a one-dimensional DP over prefix lengths, made fast by a **trie**. Let $dp[i]$ be the number of ways to form the first i characters; each way ends in some dictionary word.

State and how the trie speeds it

$dp[i]$ = ways to form $s[0..i-1]$, with $dp[0] = 1$. From a position i with $dp[i] > 0$, walk down a trie of the dictionary following $s[i], s[i+1], \dots$; whenever the walk reaches a node marking the end of a word (say the word spans $s[i..j-1]$), that word can extend every way of forming the first i characters, so $dp[j] += dp[i]$. The trie lets us discover all dictionary words starting at i in time proportional to the longest such word, without testing each dictionary word separately.

Push Contributions Forward Along the Trie

Rather than asking “which words end at j ?” we push: from each i we follow the single path $s[i..]$ in the trie and deposit $dp[i]$ into $dp[j]$ at every word-ending node. Each character of s is followed once per starting position, but the walk stops as soon as the trie path dies.

Algorithm

1. Insert all dictionary words into a trie, marking word ends.
2. $dp[0] = 1$. For $i = 0 \dots n-1$ with $dp[i] > 0$: walk the trie along $s[i..]$; at each word-end reached at column j , add $dp[i]$ to $dp[j]$.
3. Answer $dp[n]$.

Dry run

Dry Run. $s = \text{“ababc”}$; dictionary $\{\text{“ab”}, \text{“abab”}, \text{“c”}\}$.

- $dp[0] = 1$. From 0: “ab” ends at 2 $\Rightarrow dp[2] += 1$; “abab” ends at 4 $\Rightarrow dp[4] += 1$.
- From 2 ($dp = 1$): “ab” ends at 4 $\Rightarrow dp[4] = 2$.
- From 4 ($dp = 2$): “c” ends at 5 $\Rightarrow dp[5] = 2$. Answer 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int MOD = 1e9 + 7;
4
5 struct Trie {
6     struct Node { int nxt[26]; bool end; Node() { memset(nxt, -1, sizeof
7         nxt); end=false; } };
8     vector<Node> t{Node()};
9     void insert(const string& w) {

```

```

9      int cur = 0;
10     for (char c : w) {
11         int x = c - 'a';
12         if (t[cur].nxt[x] == -1) { t[cur].nxt[x] = t.size(); t.
            push_back(Node()); }
13         cur = t[cur].nxt[x];
14     }
15     t[cur].end = true;
16 }
17 };
18
19 int main() {
20     string s; int k;
21     cin >> s >> k;
22     Trie tr;
23     for (int i = 0; i < k; i++) { string w; cin >> w; tr.insert(w); }
24
25     int n = s.size();
26     vector<long long> dp(n + 1, 0);
27     dp[0] = 1;
28     for (int i = 0; i < n; i++) {
29         if (!dp[i]) continue;
30         int cur = 0;
31         for (int j = i; j < n; j++) {
32             int x = s[j] - 'a';
33             if (tr.t[cur].nxt[x] == -1) break;           // no word
34                                                         continues
35             cur = tr.t[cur].nxt[x];
36             if (tr.t[cur].end) dp[j + 1] = (dp[j + 1] + dp[i]) % MOD;
37         }
38     }
39     cout << dp[n] << "\n";
40     return 0;
}

```

Complexity Analysis

Time Complexity. $O(n \cdot L)$ where L is the longest word (each start walks at most L trie steps). **Space Complexity.** $O(\text{total dictionary length} \cdot 26)$ for the trie plus $O(n)$ DP.

Key Takeaway

Segmentation counting is prefix DP; a trie turns “which words start here?” into a single character-by-character walk. The same push-forward structure solves word-break existence, minimum-word segmentation, and tokenisation.

1.3 String Matching

Problem Statement. Given a text t and a pattern p , count how many times p occurs in t (occurrences may overlap).

Pattern Recognition

“Count exact occurrences of a pattern” is solved in linear time by the **Z-function** (or KMP). Concatenate $p + \# + t$ with a separator $\#$ not in either string; a position in the t -part whose Z-value equals $|p|$ is an occurrence.

The Z-function

The Z-function of a string w gives, for each index i , the length of the longest substring starting at i that matches a prefix of w . Build $w = p + \# + t$. Wherever $Z[i] = |p|$ and i lies in the t portion, the text matches the pattern's full length at that spot – an occurrence. The separator $\#$ guarantees a match cannot span the boundary. The Z-array itself is computed in $O(|w|)$ using a maintained “Z-box” – the rightmost match interval seen so far.

Reuse Previous Matches via the Z-box

When computing $Z[i]$, if i falls inside a previously found match interval $[l, r]$, the value $Z[i - l]$ is already known and can seed $Z[i]$, so each character is compared a constant number of times amortised. This is what makes Z (and KMP) linear rather than quadratic.

Algorithm

1. Form $w = p + \# + t$ and compute its Z-array.
2. Count indices i (in the t region) with $Z[i] = |p|$.
3. Output the count.

Dry run

Dry Run. $t = \text{“saippua”}$, $p = \text{“pp”}$; $w = \text{“pp\#saippua”}$.

- $|p| = 2$. Scanning w , the substring “pp” inside t starts at one position; there $Z = 2$.
- Exactly one index has $Z = 2$, so the pattern occurs once.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> zFunction(const string& w) {
5      int n = w.size();
6      vector<int> z(n, 0);
7      z[0] = n;
8      int l = 0, r = 0;
9      for (int i = 1; i < n; i++) {
10         if (i < r) z[i] = min(r - i, z[i - l]);
11         while (i + z[i] < n && w[z[i]] == w[i + z[i]]) z[i]++;
12         if (i + z[i] > r) { l = i; r = i + z[i]; }
13     }
14     return z;
15 }
16
17 int main() {
18     string t, p;
19     cin >> t >> p;
20     string w = p + "#" + t;
21     vector<int> z = zFunction(w);
22
23     int count = 0, m = p.size();
24     for (int i = m + 1; i < (int)w.size(); i++)    // t region
25         if (z[i] == m) count++;

```

```

26     cout << count << "\n";
27     return 0;
28 }

```

Complexity Analysis

Time Complexity. $O(|p| + |t|)$ – one linear Z-array pass. **Space Complexity.** $O(|p| + |t|)$ for the combined string and its Z-array.

Key Takeaway

Exact pattern counting reduces to “where does a prefix of $p + \# + t$ reappear at full pattern length,” which the Z-function answers in linear time. KMP’s prefix function gives an equivalent solution; keep both in your toolkit as interchangeable linear matchers.

1.4 Finding Borders

Problem Statement. A border of a string is a proper prefix that is also a suffix (shorter than the whole string). Given a string, output the lengths of all its borders in increasing order.

Pattern Recognition

“All borders of a string” is a direct application of the **prefix function** (KMP failure function). The longest border of the whole string is $\pi[n - 1]$; following the “failure chain” $\pi[n - 1] \rightarrow \pi[\pi[n - 1] - 1] \rightarrow \dots$ enumerates every border.

Prefix function and the failure chain

The prefix function $\pi[i]$ is the length of the longest proper border of the prefix $s[0..i]$. A key fact: the set of all border lengths of the *whole* string is exactly $\{\pi[n - 1], \pi[\pi[n - 1] - 1], \dots\}$ down to 0. This is because the longest border’s own longest border is the next-longest border of the whole string, and so on. Collecting the chain and reversing it yields all borders in increasing order.

Borders Nest Like Russian Dolls

Every border of s is itself bordered by the next-shorter border. So the failure chain from $\pi[n - 1]$ visits all of them in decreasing order – each obtained by looking up the prefix function at the previous border’s last index.

Algorithm

1. Compute the prefix function π of s .
2. Starting from $b = \pi[n - 1]$, repeatedly record b and set $b = \pi[b - 1]$ until $b = 0$.
3. Reverse the collected lengths and output them (increasing order).

Dry run

Dry Run. $s = \text{“ab cab ab cab”}$.

- The whole string’s longest border is “ab cab” (length 5): $\pi[9] = 5$.
- $\pi[4] = 2$ (border “ab”); $\pi[1] = 0$. Chain: 5, 2.

- Increasing order: 2, 5 – the border lengths.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> prefixFunction(const string& s) {
5     int n = s.size();
6     vector<int> pi(n, 0);
7     for (int i = 1; i < n; i++) {
8         int j = pi[i - 1];
9         while (j > 0 && s[i] != s[j]) j = pi[j - 1];
10        if (s[i] == s[j]) j++;
11        pi[i] = j;
12    }
13    return pi;
14 }
15
16 int main() {
17     string s;
18     cin >> s;
19     int n = s.size();
20     vector<int> pi = prefixFunction(s);
21
22     vector<int> borders;
23     for (int b = pi[n - 1]; b > 0; b = pi[b - 1]) borders.push_back(b);
24     reverse(borders.begin(), borders.end());
25
26     for (int b : borders) cout << b << " ";
27     cout << "\n";
28     return 0;
29 }

```

Complexity Analysis

Time Complexity. $O(n)$ for the prefix function; the failure chain has $O(n)$ total length.
Space Complexity. $O(n)$.

Key Takeaway

Borders are the prefix function's reason for existing. Walking the failure chain from $\pi[n-1]$ lists them all. This chain is also how KMP recovers after a mismatch and how periods (next section) are derived.

1.5 Finding Periods

Problem Statement. A period of a string s of length n is an integer p ($1 \leq p \leq n$) such that $s[i] = s[i + p]$ for all valid i (equivalently, s is a prefix of an infinite repetition of its first p characters). Output all periods in increasing order.

Pattern Recognition

Periods and borders are two sides of one coin: **p is a period of s if and only if s has a border of length $n - p$.** So compute the prefix function, enumerate all border lengths b

(including 0), and each gives a period $n - b$.

The period–border duality

If s has a border of length b , the first $n - b$ characters repeat to cover the string, making $n - b$ a period; conversely every period p leaves a border of length $n - p$. The border lengths of the whole string are the failure chain $\pi[n - 1], \pi[\pi[n - 1] - 1], \dots$ together with 0 (the empty border, which corresponds to the trivial period $p = n$). Mapping each border length b to $n - b$ and sorting gives all periods.

Every Border Length b Gives Period $n - b$

Include the empty border ($b = 0$) so that $p = n$ (the whole string is always its own period) is present. The failure chain supplies the remaining borders. One prefix-function pass yields every period.

Algorithm

1. Compute the prefix function π .
2. Collect border lengths via the failure chain from $\pi[n - 1]$, plus 0.
3. For each border length b , output $n - b$; sort ascending.

Dry run

Dry Run. $s = \text{"abcbca"} \ (n = 7)$.

- Longest border “abca” has length 4; chain gives borders 4, 1; plus 0.
- Periods: $7 - 4 = 3$, $7 - 1 = 6$, $7 - 0 = 7$.
- Increasing: 3, 6, 7 (indeed “abc” repeats with period 3).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> prefixFunction(const string& s) {
5     int n = s.size();
6     vector<int> pi(n, 0);
7     for (int i = 1; i < n; i++) {
8         int j = pi[i - 1];
9         while (j > 0 && s[i] != s[j]) j = pi[j - 1];
10        if (s[i] == s[j]) j++;
11        pi[i] = j;
12    }
13    return pi;
14 }
15
16 int main() {
17     string s;
18     cin >> s;
19     int n = s.size();

```

```

20     vector<int> pi = prefixFunction(s);
21
22     vector<int> periods;
23     for (int b = pi[n - 1]; b > 0; b = pi[b - 1]) periods.push_back(n -
24         b);
25     periods.push_back(n); // border 0 -> period
26                             n
27     sort(periods.begin(), periods.end());
28
29     for (int p : periods) cout << p << " ";
30     cout << "\n";
31     return 0;
32 }

```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(n)$.

Key Takeaway

Period equals n minus a border length. Once you see that borders and periods are complementary, every “how does this string repeat?” question becomes a prefix-function query. This duality also underlies detecting the smallest repeating block of a string.

1.6 Minimal Rotation

Problem Statement. Given a string s , consider all n rotations (cyclic shifts) of it. Output the lexicographically smallest rotation.

Pattern Recognition

“Lexicographically smallest cyclic rotation” has a linear-time solution – **Booth’s algorithm** (or the equivalent least-rotation scan) – that finds the starting index of the minimal rotation by comparing candidate starts and skipping ranges that cannot win.

The least-rotation scan

Work on $s + s$ (so every rotation is a length- n window). Maintain two candidate start positions i and j and an offset k : compare $s[i + k]$ with $s[j + k]$. If they are equal, extend k . If one is smaller, the losing candidate – and, crucially, *every start it skipped over* – can be discarded by jumping it past $j + k$ (or $i + k$). Because each comparison either advances k or eliminates a range of candidates, the total work is linear. The surviving candidate is the minimal rotation’s start.

Skip Doomed Starts in Bulk

When candidate j beats candidate i after matching k characters, none of the starts strictly between i and $i + k + 1$ can be minimal either (they were dominated), so we jump i past them all. This bulk elimination is what keeps the scan $O(n)$ despite comparing rotations.

Algorithm

1. Set $i = 0$, $j = 1$, $k = 0$ over the doubled string $ss = s + s$.

- While all three are in range, compare $ss[i + k]$ and $ss[j + k]$: equal $\Rightarrow k++$; otherwise advance the larger candidate past $\max(i, j) + k + 1$, reset $k = 0$, and keep $i < j$.
- The answer starts at $\min(i, j)$; output $ss[\text{start} .. \text{start} + n - 1]$.

Dry run

Dry Run. $s = \text{"acab"}$.

- Rotations: "acab", "caba", "abac", "baca".
- Lexicographically smallest is "abac", starting at index 2.
- The scan converges on $\text{start} = 2$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int leastRotation(const string& s) {
5     string ss = s + s;
6     int n = ss.size();
7     int i = 0, j = 1, k = 0;
8     while (i < (int)s.size() && j < (int)s.size() && k < (int)s.size())
9     {
10         char a = ss[i + k], b = ss[j + k];
11         if (a == b) { k++; }
12         else if (a > b) { i = max(i + k + 1, j + 1); k = 0; }
13         else { j = max(j + k + 1, i + 1); k = 0; }
14     }
15     return min(i, j); // start of minimal
16                       rotation
17 }
18
19 int main() {
20     string s;
21     cin >> s;
22     int start = leastRotation(s);
23     string ss = s + s;
24     cout << ss.substr(start, s.size()) << "\n";
25     return 0;
26 }
```

Complexity Analysis

Time Complexity. $O(n)$ – each comparison advances k or a candidate pointer, and both are bounded by $O(n)$ total. **Space Complexity.** $O(n)$ for the doubled string.

Key Takeaway

The least-rotation scan finds the minimal cyclic shift in linear time by eliminating dominated starts in bulk. It is the canonical tool for canonicalising cyclic strings – useful for necklace counting and comparing circular sequences up to rotation.

1.7 Longest Palindrome

Problem Statement. Given a string, find its longest substring that is a palindrome (reads the same forwards and backwards). Output that substring.

Pattern Recognition

“Longest palindromic substring” is exactly what **Manacher’s algorithm** computes in $O(n)$: for every possible centre it finds the radius of the longest palindrome centred there, reusing already-known radii via a maintained “current rightmost palindrome.”

Manacher in one idea

For each centre, a palindrome extends symmetrically until the characters disagree. The naive approach re-expands every centre from scratch ($O(n^2)$). Manacher keeps the palindrome with the farthest right edge seen so far, centred at c with right boundary r . For a new centre $i < r$, its mirror $2c - i$ already has a known radius, which can seed i ’s radius (bounded by $r - i$) before any character comparison – so each character is matched only a constant number of times amortised. Handling even-length palindromes is done either with a second pass or by inserting separators.

Mirror Symmetry Seeds Each New Centre

Inside the current rightmost palindrome, position i mirrors $2c - i$, so its radius starts at least as large as the mirror’s (capped by the boundary). Only growth beyond the boundary requires real comparisons, which advance r – giving linear total work.

Algorithm

1. Transform s to t by inserting a sentinel (e.g. $\#$) between every character and at the ends, so all palindromes become odd-length.
2. Compute radii $P[i]$ over t using the mirror-seed rule and a maintained (c, r) .
3. The maximum $P[i]$ locates the longest palindrome; map back to indices in s and output the substring.

Dry run

Dry Run. $s = \text{“aybabtu”}$.

- Centred at “bab”, the palindrome extends to radius covering “bab”.
- No longer palindrome exists, so the answer is “bab” (length 3).

C++ implementation

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string s;
6     cin >> s;
7     // transform: ^ # a # b # ... $    (sentinels avoid bounds checks)
8     string t = "^#";
```

```

9   for (char c : s) { t += '#'; t += c; }
10  t += "$";
11
12  int n = t.size();
13  vector<int> P(n, 0);
14  int c = 0, r = 0, bestLen = 0, bestCenter = 0;
15  for (int i = 1; i < n - 1; i++) {
16      if (i < r) P[i] = min(r - i, P[2 * c - i]); // mirror seed
17      while (t[i + P[i] + 1] == t[i - P[i] - 1]) P[i]++;
18      if (i + P[i] > r) { c = i; r = i + P[i]; }
19      if (P[i] > bestLen) { bestLen = P[i]; bestCenter = i; }
20  }
21  int start = (bestCenter - bestLen) / 2; // map back to s
22  cout << s.substr(start, bestLen) << "\n";
23  return 0;
24 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each character is matched a constant number of times amortised. **Space Complexity.** $O(n)$ for the transformed string and radius array.

Key Takeaway

Manacher computes every palindromic radius in linear time by seeding new centres from their mirrors within the current rightmost palindrome. The sentinel transform unifies odd and even cases. It is the definitive tool for palindrome-substring problems, reused in the next section to count them all.

1.8 All Palindromes

Problem Statement. Given a string, count the total number of palindromic substrings (each occurrence counted, for every start/end pair that forms a palindrome).

Pattern Recognition

“Count all palindromic substrings” is again **Manacher’s algorithm**: the radius at each centre directly gives how many palindromes are centred there, so summing the radii counts every palindromic substring in $O(n)$.

Radii are counts

For an odd centre at character i with maximal radius $d_1[i]$ (counting the centre itself), there are exactly $d_1[i]$ odd-length palindromes centred at i : lengths $1, 3, \dots, 2d_1[i] - 1$. Likewise each even centre between two characters contributes $d_2[i]$ even-length palindromes. So the total number of palindromic substrings is $\sum_i d_1[i] + \sum_i d_2[i]$. Manacher computes both radius arrays in linear time, and the sum is the answer.

Every Nested Palindrome Counts Once

A palindrome of radius k around a centre contains, at the same centre, palindromes of every smaller radius – k of them. Adding the radius at each centre therefore counts every (centre, length) palindrome exactly once, with no double counting across centres.

Algorithm

1. Run Manacher on the sentinel-transformed string to get radii $P[i]$.
2. In the transformed indexing, each centre contributes $\lceil P[i]/2 \rceil$ palindromic substrings; sum over all centres (a single combined pass counts both odd and even).
3. Output the total.

Dry run

Dry Run. $s = \text{"aaa"}$.

- Single characters: 3 palindromes. Length-2 “aa”: 2. Length-3 “aaa”: 1.
- Total = $3 + 2 + 1 = 6$, which equals the sum of Manacher radii.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string s;
6     cin >> s;
7     string t = "^";
8     for (char c : s) { t += '#'; t += c; }
9     t += "$";
10
11     int n = t.size();
12     vector<int> P(n, 0);
13     int c = 0, r = 0;
14     long long total = 0;
15     for (int i = 1; i < n - 1; i++) {
16         if (i < r) P[i] = min(r - i, P[2 * c - i]);
17         while (t[i + P[i] + 1] == t[i - P[i] - 1]) P[i]++;
18         if (i + P[i] > r) { c = i; r = i + P[i]; }
19         total += (P[i] + 1) / 2;           // palindromes at
20                                         // this center
21     }
22     cout << total << "\n";
23     return 0;
24 }
```

Complexity Analysis

Time Complexity. $O(n)$ – one Manacher pass. **Space Complexity.** $O(n)$ for the transformed string and radii.

Key Takeaway

The radius Manacher computes at each centre *is* the count of palindromes there, so counting all palindromic substrings is a single sum. For counting *distinct* palindromes instead, an Eertree (palindromic tree) is the specialised structure – but for total occurrences, Manacher suffices.

1.9 Required Substring

Problem Statement. Count the number of strings of length n over the lowercase alphabet (26 letters) that contain a given pattern p as a substring. Output the count modulo $10^9 + 7$.

Pattern Recognition

“Count length- n strings that **contain** a pattern” is easier via the complement: count strings that *avoid* p , then subtract from 26^n . Avoidance is tracked by a DP running the **KMP automaton** – the state is how much of p is currently matched.

DP over KMP automaton states

Let $m = |p|$. Build the KMP automaton: from a state j (meaning the last j characters matched a prefix of p) reading character c , the automaton moves to $\delta(j, c)$, the new longest matched prefix. Define $dp[i][j]$ = number of length- i strings whose automaton state is j *without ever reaching state m* . Transition over all 26 characters:

$$dp[i+1][\delta(j, c)] += dp[i][j], \quad \text{only when } \delta(j, c) < m.$$

Reaching state m means p appeared, so those transitions are dropped from the “avoid” count. The answer is $26^n - \sum_{j < m} dp[n][j]$.

Match Progress Is the Only State That Matters

Whether a string avoids p depends solely on how much of p is currently matched – the KMP state. Building the automaton once ($O(26m)$) lets the DP forget the actual characters and track just the m possible progress levels.

Algorithm

1. Compute p ’s prefix function and the transition $\delta(j, c)$ for $j = 0 \dots m-1$, $c \in \{a..z\}$.
2. $dp[0][0] = 1$. For $i = 0 \dots n-1$, push each state over all 26 characters, discarding transitions into state m .
3. Answer $26^n - \sum_{j=0}^{m-1} dp[n][j]$ modulo $10^9 + 7$.

Dry run

Dry Run. $n = 2$, $p = \text{“aa”}$; alphabet size 2 for illustration ($\{a, b\}$).

- Strings of length 2: aa, ab, ba, bb . Only aa contains “aa”.
- Avoidance DP counts 3 (ab, ba, bb); total $2^2 - 3 = 1$. With the real 26-letter alphabet the same logic gives the count.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const long long MOD = 1e9 + 7;
4
5 int main() {
6     int n;

```

```

7   string p;
8   cin >> n >> p;
9   int m = p.size();
10
11  vector<int> pi(m, 0);
12  for (int i = 1; i < m; i++) {
13      int j = pi[i - 1];
14      while (j > 0 && p[i] != p[j]) j = pi[j - 1];
15      if (p[i] == p[j]) j++;
16      pi[i] = j;
17  }
18  // automaton transitions delta[j][c], j in 0..m
19  vector<array<int,26>> delta(m + 1);
20  for (int j = 0; j <= m; j++)
21      for (int c = 0; c < 26; c++) {
22          if (j < m && c == p[j] - 'a') delta[j][c] = j + 1;
23          else delta[j][c] = (j == 0) ? 0 : delta[pi[j - 1]][c];
24      }
25
26  vector<vector<long long>> dp(n + 1, vector<long long>(m + 1, 0));
27  dp[0][0] = 1;
28  for (int i = 0; i < n; i++)
29      for (int j = 0; j < m; j++) if (dp[i][j])
30          for (int c = 0; c < 26; c++) {
31              int nj = delta[j][c];
32              if (nj < m) dp[i + 1][nj] = (dp[i + 1][nj] + dp[i][j])
33                  % MOD;
34          }
35
36  long long total = 1; // 26^n
37  for (int i = 0; i < n; i++) total = total * 26 % MOD;
38  long long avoid = 0;
39  for (int j = 0; j < m; j++) avoid = (avoid + dp[n][j]) % MOD;
40  cout << ((total - avoid) % MOD + MOD) % MOD << "\n";
41  return 0;
}

```

Complexity Analysis

Time Complexity. $O(n \cdot m \cdot 26)$ for the DP, plus $O(m \cdot 26)$ to build the automaton. **Space Complexity.** $O(n \cdot m)$ (reducible to $O(m)$ with rolling rows).

Key Takeaway

“Contains a pattern” is best counted as 26^n minus “avoids the pattern,” and avoidance is a DP over KMP-automaton states. Turning a pattern into an automaton and running a counting DP on its states is a recurring competitive-programming template.

1.10 Palindrome Queries

Problem Statement. Given a string, process two kinds of queries: (1) change the character at position k ; (2) ask whether the substring $[a, b]$ is a palindrome. Answer all type-2 queries online.

Pattern Recognition

“Update a character, then test a substring for palindrome” combines point updates with range equality checks – exactly what **polynomial hashing over a Fenwick tree** supports. Maintain a forward hash and a reversed hash; the substring is a palindrome iff the two agree.

Two hashes, both point-updatable

Store the string’s characters weighted by powers of a base B in two Fenwick (BIT) trees: one for the forward direction, one for the reversed direction. The hash of $[a, b]$ read forwards is a normalised range sum from the forward BIT; the hash of $[a, b]$ read backwards is the corresponding range sum from the reversed BIT. Normalising both to the same starting power lets them be compared directly. A character update is two point updates (one per tree). The substring is a palindrome exactly when the forward and reversed hashes match.

Palindrome = Forward Hash Equals Reversed Hash

A substring reads the same both ways iff its forward and backward hashes coincide. Encoding each character as $c \cdot B^{\text{pos}}$ in a BIT makes both hashes range sums, so point updates and range queries are each $O(\log n)$. Use two independent moduli/bases to make hash collisions negligible.

Algorithm

1. Precompute powers of B ; insert each character into a forward and a reversed Fenwick tree.
2. Update: point-update both trees at the changed position.
3. Query $[a, b]$: fetch the forward and reversed range hashes, normalise to a common power, compare; equal $\Rightarrow$ palindrome.

Dry run

Dry Run. $s = \text{“aybabtu”}$; query palindrome on “bab” $([2, 4])$.

- Forward hash of “bab” and reversed hash of “bab” both encode b, a, b in mirror order and match.
- After updating position 3 ($\text{‘a’} \rightarrow \text{‘x’}$), “bxb” still matches (palindrome); “bxa” would not.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef unsigned long long ull;
4 const ull B = 131;
5
6 struct BIT {
7     int n; vector<ull> tree;
8     BIT(int n) : n(n), tree(n + 1, 0) {}
9     void update(int i, ull val) { for (++i; i <= n; i += i & -i) tree[i]
10         += val; }
11     ull query(int i) { ull s = 0; for (++i; i > 0; i -= i & -i) s +=
12         tree[i]; return s; }

```

```

11     ull range(int l, int r) { return query(r) - (l ? query(l - 1) : 0);
12     }
13 };
14 int main() {
15     int n, q;
16     string s;
17     cin >> n >> q >> s;
18     vector<ull> pw(n + 1, 1);
19     for (int i = 1; i <= n; i++) pw[i] = pw[i - 1] * B;
20
21     BIT fwd(n), rev(n);
22     for (int i = 0; i < n; i++) {
23         fwd.update(i, (ull)s[i] * pw[i]);           // weight by
24         rev.update(i, (ull)s[i] * pw[n - 1 - i]);   position
25     }
26
27     while (q--) {
28         int type; cin >> type;
29         if (type == 1) {
30             int k; char c; cin >> k >> c; k--;
31             fwd.update(k, ((ull)c - s[k]) * pw[k]);
32             rev.update(k, ((ull)c - s[k]) * pw[n - 1 - k]);
33             s[k] = c;
34         } else {
35             int a, b; cin >> a >> b; a--; b--;
36             ull h1 = fwd.range(a, b) * pw[n - 1 - b]; //
37             ull h2 = rev.range(a, b) * pw[a];          normalise
38             cout << (h1 == h2 ? "YES" : "NO") << "\n";
39         }
40     }
41     return 0;
42 }

```

Complexity Analysis

Time Complexity. $O((n + q) \log n)$ – each update and query is $O(\log n)$. **Space Complexity.** $O(n)$ for the two Fenwick trees and power table.

Key Takeaway

Dynamic substring equality is polynomial hashing plus a point-updatable Fenwick tree; a palindrome test is just “forward hash equals reversed hash.” For safety against adversarial collisions, run two independent hashes. This forward/reversed-hash pattern answers many online substring comparison queries.

1.11 Finding Patterns

Problem Statement. Given a text t and k patterns, determine for each pattern whether it appears as a substring of t . Output “YES” or “NO” per pattern.

Pattern Recognition

“Do these many patterns occur in one fixed text?” is the signature use of a **suffix automaton (SAM)** built on t . The SAM recognises exactly the substrings of t , so each pattern is a matter of walking its characters through the automaton.

The suffix automaton recognises every substring

A suffix automaton of t is the smallest automaton whose paths from the initial state spell precisely the substrings of t . Building it takes $O(|t|)$ (with a constant or $\log \Sigma$ factor for transitions). To test a pattern p , start at the initial state and follow the transition for each character of p ; if at any point the needed transition is missing, p is not a substring, otherwise it is. The total query cost is the sum of pattern lengths.

Build Once, Query Many

The SAM front-loads all the work into a single linear construction over t ; afterwards every pattern is answered in time proportional to its own length. This “preprocess the text, stream the queries” shape is why SAM dominates multi-pattern substring questions.

Algorithm

1. Build the suffix automaton of t by extending it one character at a time.
2. For each pattern, walk from the initial state following its characters; report YES if the walk completes, else NO.

Dry run

Dry Run. $t = \text{“aybabtu”}$; patterns “bab”, “ba”, “xyz”.

- “bab”: walk $b \rightarrow a \rightarrow b$ succeeds – YES.
- “ba”: succeeds – YES.
- “xyz”: the transition on “x” is missing at the start – NO.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SAM {
5     struct State { int len, link; map<char,int> next; };
6     vector<State> st;
7     int last;
8     SAM() { st.push_back({0, -1, {}}); last = 0; }
9     void extend(char c) {
10         int cur = st.size();
11         st.push_back({st[last].len + 1, -1, {}});
12         int p = last;
13         while (p != -1 && !st[p].next.count(c)) { st[p].next[c] = cur;
14             p = st[p].link; }
15         if (p == -1) st[cur].link = 0;
16         else {

```

```

16         int q = st[p].next[c];
17         if (st[p].len + 1 == st[q].len) st[cur].link = q;
18         else {
19             int clone = st.size();
20             st.push_back(st[q]);
21             st[clone].len = st[p].len + 1;
22             while (p != -1 && st[p].next[c] == q) { st[p].next[c] =
23                 clone; p = st[p].link; }
24             st[q].link = st[cur].link = clone;
25         }
26         last = cur;
27     }
28 };
29
30 int main() {
31     string t;
32     int k;
33     cin >> t >> k;
34     SAM sam;
35     for (char c : t) sam.extend(c);
36
37     while (k--) {
38         string p;
39         cin >> p;
40         int cur = 0;
41         bool ok = true;
42         for (char c : p) {
43             auto it = sam.st[cur].next.find(c);
44             if (it == sam.st[cur].next.end()) { ok = false; break; }
45             cur = it->second;
46         }
47         cout << (ok ? "YES" : "NO") << "\n";
48     }
49     return 0;
50 }

```

Complexity Analysis

Time Complexity. $O(|t| \log \Sigma)$ to build, plus $O(\sum |p_i| \log \Sigma)$ for all queries. **Space Complexity.** $O(|t|)$ states.

Key Takeaway

A suffix automaton turns “is this a substring?” into a walk, so many patterns against a fixed text are answered by one construction plus cheap traversals. The same SAM, augmented with counts, answers the next two problems (Counting Patterns, Pattern Positions) with almost no extra code.

1.12 Counting Patterns

Problem Statement. Given a text t and k patterns, count for each pattern how many times it occurs in t (occurrences may overlap).

Pattern Recognition

This is Finding Patterns with counts: the number of occurrences of a substring equals the size of its **endpos** set, which the suffix automaton stores per state. Compute those sizes once, then each pattern's count is a lookup after walking to its state.

Endpos sizes count occurrences

In a suffix automaton, each state represents a set of substrings that share the same set of ending positions (their *endpos* set). The number of occurrences of any substring in a state equals that state's endpos-set size. To compute these: mark each state created as a genuine prefix-endpoint (the "*last*" after each extension) with count 1 and clones with 0, then propagate counts up the suffix-link tree in order of decreasing length – each state adds its count to its link. A pattern's count is the endpos size of the state its walk ends in.

Propagate Counts Up the Link Tree

The suffix-link tree groups substrings by endpos containment: a child's occurrences are a subset that also count toward the parent. Summing counts from longer states into their links (processing by decreasing *len*) computes every endpos size in one linear pass.

Algorithm

1. Build the SAM of t ; set $cnt = 1$ for each non-clone state (the *last* after each extend), 0 for clones.
2. Sort states by *len* descending; add each state's cnt to its suffix link's cnt .
3. For each pattern, walk to its state; output that state's cnt (or 0 if the walk fails).

Dry run

Dry Run. $t = \text{"aaa"}$; pattern "a".

- "a" occurs at positions 1, 2, 3 – endpos size 3.
- The SAM state for "a" accumulates count 3 after propagation, which is the reported answer.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SAM {
5     struct State { int len, link; map<char,int> next; };
6     vector<State> st; vector<long long> cnt; int last;
7     SAM() { st.push_back({0, -1, {}}); cnt.push_back(0); last = 0; }
8     void extend(char c) {
9         int cur = st.size();
10        st.push_back({st[last].len + 1, -1, {}}); cnt.push_back(1); //
            // real endpoint
11        int p = last;
12        while (p != -1 && !st[p].next.count(c)) { st[p].next[c] = cur;
13            p = st[p].link; }
14        if (p == -1) st[cur].link = 0;

```

```

14     else {
15         int q = st[p].next[c];
16         if (st[p].len + 1 == st[q].len) st[cur].link = q;
17         else {
18             int clone = st.size();
19             st.push_back(st[q]); cnt.push_back(0); //
20             clone: no endpoint
21             st[clone].len = st[p].len + 1;
22             while (p != -1 && st[p].next[c] == q) { st[p].next[c] =
23                 clone; p = st[p].link; }
24             st[q].link = st[cur].link = clone;
25         }
26     }
27     last = cur;
28 }
29
30 int main() {
31     string t; int k;
32     cin >> t >> k;
33     SAM sam;
34     for (char c : t) sam.extend(c);
35
36     int m = sam.st.size();
37     vector<int> order(m);
38     for (int i = 0; i < m; i++) order[i] = i;
39     sort(order.begin(), order.end(),
40         [&](int a, int b){ return sam.st[a].len > sam.st[b].len; });
41     for (int v : order)
42         if (sam.st[v].link >= 0) sam.cnt[sam.st[v].link] += sam.cnt[v];
43
44     while (k--) {
45         string p; cin >> p;
46         int cur = 0; bool ok = true;
47         for (char c : p) {
48             auto it = sam.st[cur].next.find(c);
49             if (it == sam.st[cur].next.end()) { ok = false; break; }
50             cur = it->second;
51         }
52         cout << (ok ? sam.cnt[cur] : 0) << "\n";
53     }
54     return 0;
}

```

Complexity Analysis

Time Complexity. $O(|t| \log \Sigma)$ build and count propagation, plus $O(\sum |p_i| \log \Sigma)$ queries.

Space Complexity. $O(|t|)$.

Key Takeaway

Occurrence count = endpos-set size, computed by propagating counts up the suffix-link tree. This single augmentation upgrades the SAM from “does it occur?” to “how many times?” – and the same endpos machinery drives distinct-substring counting and ranking later in the chapter.

1.13 Pattern Positions

Problem Statement. Given a text t and k patterns, output for each pattern the starting position of its *first* occurrence in t (1-indexed), or -1 if it does not occur.

Pattern Recognition

Same suffix automaton, one more annotation: store, for each state, the ending position of the **first** occurrence of its substrings (*firstpos*). A pattern's first-occurrence start is then $firstpos - |p| + 1$.

Tracking the first endpos

When the automaton is extended with the character at index i (1-indexed), the newly created state's longest substring first *ends* at position i , so set its $firstpos = i$. A cloned state inherits the $firstpos$ of the state it was split from, because the clone represents shorter substrings that first appear no later. After building, a pattern p that walks to state v has its earliest occurrence ending at $firstpos[v]$, hence starting at $firstpos[v] - |p| + 1$.

The First Endpoint Locates the First Occurrence

Every substring in a state first occurs at the same earliest ending position (that is what an endpos class guarantees for its shortest common context). Recording that position at creation time – and copying it to clones – gives $O(1)$ first-occurrence lookups after the walk.

Algorithm

1. Build the SAM; when creating the state at step i , set $firstpos = i$; for a clone, copy the original's $firstpos$.
2. For each pattern, walk to its state v ; if the walk fails, output -1 ; else output $firstpos[v] - |p| + 1$.

Dry run

Dry Run. $t = \text{"aybabbtu"}; \text{pattern "ab"}$.

- “ab” first occurs ending at position 4 (characters 3–4).
- $\text{Start} = 4 - 2 + 1 = 3$, the 1-indexed first position of “ab”.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SAM {
5     struct State { int len, link, firstpos; map<char,int> next; };
6     vector<State> st; int last;
7     SAM() { st.push_back({0, -1, 0, {}}); last = 0; }
8     void extend(char c, int pos) { // pos = 1-indexed end
9         position
10         int cur = st.size();
11         st.push_back({st[last].len + 1, -1, pos, {}});
12         int p = last;

```

```

12     while (p != -1 && !st[p].next.count(c)) { st[p].next[c] = cur;
13         p = st[p].link; }
14     if (p == -1) st[cur].link = 0;
15     else {
16         int q = st[p].next[c];
17         if (st[p].len + 1 == st[q].len) st[cur].link = q;
18         else {
19             int clone = st.size();
20             st.push_back(st[q]);           // inherits firstpos
21             // of q
22             st[clone].len = st[p].len + 1;
23             while (p != -1 && st[p].next[c] == q) { st[p].next[c] =
24                 clone; p = st[p].link; }
25             st[q].link = st[cur].link = clone;
26         }
27     }
28     last = cur;
29 }
30
31 int main() {
32     string t; int k;
33     cin >> t >> k;
34     SAM sam;
35     for (int i = 0; i < (int)t.size(); i++) sam.extend(t[i], i + 1);
36
37     while (k--) {
38         string p; cin >> p;
39         int cur = 0; bool ok = true;
40         for (char c : p) {
41             auto it = sam.st[cur].next.find(c);
42             if (it == sam.st[cur].next.end()) { ok = false; break; }
43             cur = it->second;
44         }
45         cout << (ok ? sam.st[cur].firstpos - (int)p.size() + 1 : -1) <<
46             "\n";
47     }
48     return 0;
49 }

```

Complexity Analysis

Time Complexity. $O(|t| \log \Sigma)$ build, $O(\sum |p_i| \log \Sigma)$ queries. **Space Complexity.** $O(|t|)$.

Key Takeaway

Recording each state's first ending position turns the SAM into a first-occurrence locator. Combined with endpos counts (previous section), one automaton answers presence, frequency, and position – the full substring query trio.

1.14 Distinct Substrings

Problem Statement. Given a string, count the number of *distinct* non-empty substrings it contains.

Pattern Recognition

“How many distinct substrings” is a one-line consequence of the **suffix automaton**: each state represents a contiguous range of substring lengths, so the total distinct count is $\sum_{v \neq \text{root}} (\text{len}[v] - \text{len}[\text{link}[v]])$.

Why the length differences count substrings

Every substring of the string corresponds to exactly one state of the suffix automaton and one length within that state’s range. A state v represents all substrings whose lengths lie in $(\text{len}[\text{link}[v]], \text{len}[v]]$ – that is $\text{len}[v] - \text{len}[\text{link}[v]]$ distinct substrings, each occurring at v ’s endpos set. Summing this difference over all non-root states counts every distinct substring exactly once.

Each State Owns a Length Interval

The suffix automaton partitions all distinct substrings among its states, and within a state the distinct substrings are precisely those of length strictly greater than the link’s length up to the state’s own length. Summing those interval sizes is the distinct-substring count – no enumeration required.

Algorithm

1. Build the suffix automaton of the string.
2. Sum $\text{len}[v] - \text{len}[\text{link}[v]]$ over all states v except the root.
3. Output the sum.

Dry run

Dry Run. $s = \text{“aa”}$.

- Distinct substrings: “a”, “aa” – two of them.
- The SAM has states for “a” (len 1, link root len 0 $\rightarrow$ 1) and “aa” (len 2, link “a” len 1 $\rightarrow$ 1), summing to 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SAM {
5     struct State { int len, link; map<char,int> next; };
6     vector<State> st; int last;
7     SAM() { st.push_back({0, -1, {}}); last = 0; }
8     void extend(char c) {
9         int cur = st.size();
10        st.push_back({st[last].len + 1, -1, {}});
11        int p = last;
12        while (p != -1 && !st[p].next.count(c)) { st[p].next[c] = cur;
13            p = st[p].link; }
14        if (p == -1) st[cur].link = 0;
15        else {

```

```

15     int q = st[p].next[c];
16     if (st[p].len + 1 == st[q].len) st[cur].link = q;
17     else {
18         int clone = st.size();
19         st.push_back(st[q]);
20         st[clone].len = st[p].len + 1;
21         while (p != -1 && st[p].next[c] == q) { st[p].next[c] =
22             clone; p = st[p].link; }
23         st[q].link = st[cur].link = clone;
24     }
25     last = cur;
26 }
27 };
28
29 int main() {
30     string s;
31     cin >> s;
32     SAM sam;
33     for (char c : s) sam.extend(c);
34
35     long long total = 0;
36     for (int v = 1; v < (int)sam.st.size(); v++)
37         total += sam.st[v].len - sam.st[sam.st[v].link].len;
38     cout << total << "\n";
39     return 0;
40 }

```

Complexity Analysis

Time Complexity. $O(n \log \Sigma)$ to build; the sum is linear in the number of states. **Space Complexity.** $O(n)$ states.

Key Takeaway

Distinct substrings = $\sum (len[v] - len[link[v]])$. This is the cleanest demonstration that a suffix automaton compresses all $O(n^2)$ substrings into $O(n)$ states, each owning a length interval. A suffix array with LCP gives the same count as $\frac{n(n+1)}{2} - \sum LCP$.

1.15 Distinct Subsequences

Problem Statement. Given a string, count the number of distinct non-empty subsequences (characters chosen in order, not necessarily contiguous; duplicates counted once). Output modulo $10^9 + 7$.

Pattern Recognition

“Count distinct subsequences” is a linear DP over prefixes. Let $dp[i]$ be the number of distinct subsequences (including the empty one) of the first i characters; each new character doubles the count but we must subtract those double-counted by a previous identical character.

Double, then remove duplicates

$dp[i]$ = distinct subsequences of $s[0..i-1]$, with $dp[0] = 1$ (the empty subsequence). Adding character $s[i-1]$, each existing subsequence can either exclude or include the new character, so naively $dp[i] = 2 dp[i-1]$. But if $s[i-1]$ appeared before at position j (its previous occurrence, 1-indexed), the subsequences formed by appending this character duplicate exactly those counted when it last appeared – namely $dp[j-1]$ of them – so we subtract:

$$dp[i] = 2 dp[i-1] - dp[\text{last}[s_i] - 1].$$

The final answer is $dp[n] - 1$ (dropping the empty subsequence).

Subtract the Last Occurrence's Contribution

Doubling over-counts precisely the subsequences that already ended in this character the last time it appeared. Subtracting $dp[j-1]$ (the state just before the previous occurrence) removes exactly those duplicates – an inclusion–exclusion in one term.

Algorithm

1. $dp[0] = 1$; track $\text{last}[c]$ = previous index of character c (0 if none).
2. For $i = 1 \dots n$: $dp[i] = 2 dp[i-1]$; if $s[i-1]$ seen before at j , subtract $dp[j-1]$; update $\text{last}[s_{i-1}] = i$.
3. Output $(dp[n] - 1) \bmod (10^9 + 7)$.

Dry run

Dry Run. $s = \text{"aba"}$.

- $dp[0] = 1$; $dp[1] = 2$ (“”, “a”).
- $dp[2] = 2 \cdot 2 = 4$ (“”, “a”, “b”, “ab”); ‘b’ is new.
- $dp[3] = 2 \cdot 4 - dp[0] = 8 - 1 = 7$ (subtract the duplicate from the earlier ‘a’). Answer $7 - 1 = 6$ non-empty subsequences.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const long long MOD = 1e9 + 7;
4
5 int main() {
6     string s;
7     cin >> s;
8     int n = s.size();
9     vector<long long> dp(n + 1, 0);
10    dp[0] = 1;
11    vector<int> last(26, 0);           // last[c] = 1-
        indexed prev pos
12
13    for (int i = 1; i <= n; i++) {
14        dp[i] = 2 * dp[i - 1] % MOD;
15        int c = s[i - 1] - 'a';

```

```

16     if (last[c] != 0) dp[i] = (dp[i] - dp[last[c] - 1] + MOD) % MOD
17         ;
18     last[c] = i;
19 }
19 cout << (dp[n] - 1 + MOD) % MOD << "\n";           // exclude empty
20 return 0;
21 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one pass. **Space Complexity.** $O(n)$ for the DP array (reducible to $O(1)$ with rolling values plus the 26 last-occurrence markers).

Key Takeaway

Counting distinct subsequences is “double and subtract the last occurrence’s prefix count.” The subtraction is a clean inclusion–exclusion that removes exactly the repeats a duplicate character would create – a technique that reappears in distinct-subarray and distinct-path counting.

1.16 Repeating Substring

Problem Statement. Given a string, find the longest substring that occurs at least twice (occurrences may overlap). If none exists, report so; otherwise output one such longest repeating substring.

Pattern Recognition

“Longest substring occurring ≥ 2 times” falls straight out of the **suffix automaton** augmented with endpos sizes: a substring repeats iff its state’s endpos set has size ≥ 2 , so the answer is the longest such state.

Longest state with endpos size at least two

Recall each SAM state’s endpos-set size (computed by propagating counts up the suffix-link tree) is the number of occurrences of its substrings. A substring repeats exactly when that count is at least 2. Among all states with count ≥ 2 , the one with the largest *len* gives the longest repeating substring; its stored first ending position (*firstpos*) lets us extract the actual characters.

Repetition = Endpos Size ≥ 2

The suffix automaton already knows how often each substring occurs. So the longest repeating substring is simply the deepest (largest-*len*) state whose occurrence count is at least two – no binary search or hashing needed, and no collision risk.

Algorithm

1. Build the SAM with endpos counts and *firstpos* per state.
2. Over all states with count ≥ 2 , take the one with maximum *len*.
3. Output the substring ending at its *firstpos* of that length (or state that none exists if no such state).

Dry run

Dry Run. $s = \text{"aabaa"}$.

- “aa” occurs at positions ending 2 and 5 – count 2, length 2.
- “a” occurs more but is shorter. The longest repeating substring is “aa”.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SAM {
5     struct State { int len, link, firstpos; map<char,int> next; };
6     vector<State> st; vector<long long> cnt; int last;
7     SAM() { st.push_back({0, -1, 0, {}}); cnt.push_back(0); last = 0; }
8     void extend(char c, int pos) {
9         int cur = st.size();
10        st.push_back({st[last].len + 1, -1, pos, {}}); cnt.push_back(1);
11
12        int p = last;
13        while (p != -1 && !st[p].next.count(c)) { st[p].next[c] = cur;
14            p = st[p].link; }
15        if (p == -1) st[cur].link = 0;
16        else {
17            int q = st[p].next[c];
18            if (st[p].len + 1 == st[q].len) st[cur].link = q;
19            else {
20                int clone = st.size();
21                st.push_back(st[q]); cnt.push_back(0);
22                st[clone].len = st[p].len + 1;
23                while (p != -1 && st[p].next[c] == q) { st[p].next[c] =
24                    clone; p = st[p].link; }
25                st[q].link = st[cur].link = clone;
26            }
27        }
28        last = cur;
29    }
30 };
31
32 int main() {
33     string s;
34     cin >> s;
35     SAM sam;
36     for (int i = 0; i < (int)s.size(); i++) sam.extend(s[i], i + 1);
37
38     int m = sam.st.size();
39     vector<int> order(m);
40     iota(order.begin(), order.end(), 0);
41     sort(order.begin(), order.end(),
42         [&](int a, int b){ return sam.st[a].len > sam.st[b].len; });
43     for (int v : order)
44         if (sam.st[v].link >= 0) sam.cnt[sam.st[v].link] += sam.cnt[v];
45
46     int best = 0;
47     for (int v = 1; v < m; v++)

```

```

45     if (sam.cnt[v] >= 2 && sam.st[v].len > sam.st[best].len) best =
        v;
46
47     if (best == 0) cout << "-1\n";
48     else {
49         int len = sam.st[best].len, end = sam.st[best].firstpos;
50         cout << s.substr(end - len, len) << "\n";
51         // (end is 1-indexed end position; substring is s[end-len ..
            end-1])
52     }
53     return 0;
54 }

```

Complexity Analysis

Time Complexity. $O(n \log \Sigma)$ to build and propagate. **Space Complexity.** $O(n)$ states.

Key Takeaway

The longest repeating substring is the deepest suffix-automaton state with occurrence count ≥ 2 – reusing the endpos-size and first-position augmentations from earlier sections. A binary-search-plus-hashing solution also works but risks collisions; the SAM answer is exact.

1.17 String Functions

Problem Statement. Given a string, compute two classic string functions: the **prefix function** (the border/failure array) and the **Z-function**. Output both arrays.

Pattern Recognition

This problem is a direct exercise in the two linear-time fundamentals. The **prefix function** $\pi[i]$ is the longest proper border of the prefix ending at i ; the **Z-function** $z[i]$ is the longest substring starting at i that matches a prefix of the whole string.

Two views of the same self-similarity

Both functions measure how a string matches itself. The prefix function scans left to right, extending or falling back along the failure chain so each character is charged $O(1)$ amortised. The Z-function maintains the rightmost matched interval (the “Z-box”) and seeds each new index from its mirror inside that box. They are convertible into each other, but computing each directly with its own linear routine is simplest.

Prefix Function: Extend or Fall Back

At index i , try to extend the previous border: if $s[i]$ equals $s[j]$ (where $j = \pi[i - 1]$) increment, else follow $j \leftarrow \pi[j - 1]$ until a match or $j = 0$. The Z-function mirror-seeds instead. Both amortise to $O(n)$ because the fallback/expansion total is bounded by n .

Algorithm

1. Compute π : for each i , fall back via π until $s[i]$ matches or the border is empty, then set $\pi[i]$.
2. Compute z : maintain (l, r) ; seed $z[i]$ from $z[i - l]$ if inside the box, then extend by direct comparison.

3. Output both arrays.

Dry run

Dry Run. $s = \text{"abacaba"}$.

- Prefix function: $[0, 0, 1, 0, 1, 2, 3]$ – e.g. $\pi[6] = 3$ because “aba” is the longest border.
- Z-function: $[7, 0, 1, 0, 3, 0, 1]$ – $z[4] = 3$ since “aba” at index 4 matches the prefix “aba”.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> prefixFunction(const string& s) {
5     int n = s.size();
6     vector<int> pi(n, 0);
7     for (int i = 1; i < n; i++) {
8         int j = pi[i - 1];
9         while (j > 0 && s[i] != s[j]) j = pi[j - 1];
10        if (s[i] == s[j]) j++;
11        pi[i] = j;
12    }
13    return pi;
14 }
15
16 vector<int> zFunction(const string& s) {
17     int n = s.size();
18     vector<int> z(n, 0);
19     z[0] = n;
20     int l = 0, r = 0;
21     for (int i = 1; i < n; i++) {
22         if (i < r) z[i] = min(r - i, z[i - 1]);
23         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
24         if (i + z[i] > r) { l = i; r = i + z[i]; }
25     }
26     return z;
27 }
28
29 int main() {
30     string s;
31     cin >> s;
32     for (int x : prefixFunction(s)) cout << x << " ";
33     cout << "\n";
34     for (int x : zFunction(s)) cout << x << " ";
35     cout << "\n";
36     return 0;
37 }

```

Complexity Analysis

Time Complexity. $O(n)$ for each function. **Space Complexity.** $O(n)$ per array.

Key Takeaway

The prefix function and Z-function are the two workhorses of string matching; both run in linear time by never re-comparing a character more than a constant number of times amortised. Every earlier border/period/matching section is an application of one of them.

1.18 Inverse Suffix Array

Problem Statement. Given a suffix array (a permutation SA of $0 \dots n-1$ listing the starting indices of the string's suffixes in sorted order), reconstruct the *lexicographically smallest* string over the alphabet whose suffix array is exactly SA .

Pattern Recognition

“Recover a string from its suffix array” is a greedy assignment: walk the suffixes in sorted order and give each position the smallest character consistent with the ordering, incrementing the character only when two adjacent sorted suffixes force a strict increase.

When must the character increase?

Let $rank[i]$ be the position of suffix i in SA . Process consecutive suffixes $a = SA[i-1]$ and $b = SA[i]$. Suffix a must be lexicographically smaller than suffix b . Their first characters satisfy $ch[a] \leq ch[b]$. We may keep them *equal* only if the order is already settled by the following suffixes, i.e. $rank[a+1] < rank[b+1]$ (treating an out-of-range suffix as the smallest). Otherwise the first characters must differ, so we increment the current character. Assigning the smallest letter at $SA[0]$ and applying this rule yields the minimal string.

Increment Only When Forced

Keeping characters equal whenever the suffixes after them already enforce the order gives the smallest possible alphabet usage – hence the lexicographically smallest string. The next-suffix rank comparison is the exact test for “is a strict increase required here?”

Algorithm

1. Build $rank$ from SA ; define $rankNext(x) = rank[x]$ if $x < n$, else -1 .
2. Assign character 0 to $SA[0]$. For $i = 1 \dots n-1$ with $a = SA[i-1]$, $b = SA[i]$: if $rankNext(a+1) < rankNext(b+1)$ keep the current character, else increment it; assign it to b .
3. Output the characters (as 'a' + value).

Dry run

Dry Run. $n = 4$, $SA = [3, 1, 0, 2]$ (suffixes sorted).

- Assign $ch[3] = a$ (smallest, the sorted-first suffix).
- Pair (3,1): $rankNext(4) = -1 < rankNext(2)$, keep $\Rightarrow ch[1] = a$. Pair (1,0): compare $rankNext(2)$ vs $rankNext(1)$; if forced, increment.
- The construction yields the minimal string consistent with SA .

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     vector<int> sa(n);
8     for (int& x : sa) cin >> x;           // 0-indexed suffix starts
9
10    vector<int> rankArr(n);
11    for (int i = 0; i < n; i++) rankArr[sa[i]] = i;
12    auto rankNext = [&](int x) { return x < n ? rankArr[x] : -1; };
13
14    vector<int> ch(n, 0);
15    int cur = 0;
16    ch[sa[0]] = 0;
17    for (int i = 1; i < n; i++) {
18        int a = sa[i - 1], b = sa[i];
19        if (!(rankNext(a + 1) < rankNext(b + 1))) cur++;    // strict
20        // increase forced
21        ch[b] = cur;
22    }
23
24    string res;
25    for (int i = 0; i < n; i++) res += char('a' + ch[i]);
26    cout << res << "\n";
27    return 0;
28 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one pass building ranks and one pass assigning characters.

Space Complexity. $O(n)$.

Key Takeaway

Reconstructing the minimal string from a suffix array is a greedy that increments a character only when the following suffixes fail to already order two adjacent entries. This “smallest label consistent with a given order” pattern recurs whenever inverting an ordering into concrete values.

1.19 String Transform

Problem Statement. The Burrows–Wheeler transform (BWT) of a string sorts all rotations of the string (with an appended sentinel `#` that is lexicographically smallest) and outputs the last column. Given a BWT string, reconstruct the original string.

Pattern Recognition

“Invert the Burrows–Wheeler transform” is done with the **LF-mapping**: from the last column alone you can recover the first column (it is just the last column sorted), and a rank-based mapping links each character to its predecessor, letting you walk the original string out one character at a time.

LF-mapping in one idea

Let L be the given last column. The first column F is L sorted. The **LF-mapping** exploits a key invariant: the k -th occurrence of a character in L corresponds to the k -th occurrence of that same character in F . Concretely, for a row i , the character $L[i]$ maps to row $start[L[i]] + rank_i$, where $start[c]$ is the first row in F beginning with c and $rank_i$ is how many $L[i]$ appeared before index i . Following this mapping from the sentinel row spells the original string in reverse.

Same Rank in Last and First Columns

Because equal characters keep their relative order between the sorted-first and last columns, the LF-mapping is exact: it reconstructs which character precedes each one in the original. Iterating it n times recovers the whole string with no ambiguity.

Algorithm

1. Count characters of L ; compute $start[c] = \text{number of characters smaller than } c$.
2. Compute $rank_i$ for each row and the LF map $lf[i] = start[L[i]] + rank_i$.
3. Start at the sentinel row (row 0 of F), emit $L[row]$, follow lf ; after n steps reverse to get the original.

Dry run

Dry Run. Conceptually, L 's sorted copy is F ; equal letters keep order.

- From the sentinel row, each LF step jumps to the row whose first character is the current L character, at the matching rank.
- Emitting characters along this chain and reversing reproduces the original string ending in #.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      string L;
6      cin >> L;                                // BWT (last column),
7      contains '#'
8      int n = L.size();
9
10     vector<int> cnt(128, 0);
11     for (char c : L) cnt[c]++;
12     vector<int> start(128, 0);
13     for (int c = 1; c < 128; c++) start[c] = start[c - 1] + cnt[c - 1];
14
15     vector<int> lf(n);
16     vector<int> occ(128, 0);
17     for (int i = 0; i < n; i++) {
18         lf[i] = start[(int)L[i]] + occ[(int)L[i]];
19         occ[(int)L[i]]++;
20     }
21     // sentinel '#' is smallest -> first column row 0 begins with '#'

```

```

22     string res;
23     int row = 0;
24     for (int i = 0; i < n; i++) {
25         res += L[row];
26         row = lf[row];
27     }
28     reverse(res.begin(), res.end());
29     cout << res << "\n";           // original (ending
    in '#')
30     return 0;
31 }

```

Complexity Analysis

Time Complexity. $O(n + \Sigma)$ – linear counting sort plus a linear LF walk. **Space Complexity.** $O(n + \Sigma)$.

Key Takeaway

Inverting the BWT hinges on the LF-mapping’s rank-preservation between the first and last columns, letting you unwind the original one character at a time in linear time. The same rank correspondence underlies FM-index search used in bioinformatics.

1.20 Substring Order I

Problem Statement. Given a string and an index k , find the k -th *distinct* substring in lexicographic order (each distinct substring counted once).

Pattern Recognition

“ k -th distinct substring lexicographically” is a guided walk on the **suffix automaton**: precompute, for each state, how many distinct substrings can be spelled starting from it, then descend edges in alphabetical order, subtracting counts until k lands on the target.

Counting continuations per state

Every distinct substring corresponds to a unique path from the initial state along transitions. Define $d[v]$ = number of distinct substrings obtainable by continuing from state v :

$$d[v] = \sum_{(c,u) \text{ edge from } v} (1 + d[u]),$$

the 1 for the single-character extension c and $d[u]$ for everything that continues past it. Compute d in decreasing order of state length (edges only go to longer states). To find the k -th substring, start at the root and examine outgoing edges in increasing character order: an edge to u accounts for $1 + d[u]$ substrings. If $k = 1$, the current prefix plus this character is the answer; if $k \leq 1 + d[u]$, append the character, move to u , and decrement k by 1; otherwise subtract $1 + d[u]$ and try the next character.

Rank by Skipping Whole Subtrees

Because $d[u]$ counts *all* distinct substrings through edge (c, u) , we can skip an entire branch by subtracting its total when k is larger – turning a k -th-element search into an $O(\text{answer length} \times \Sigma)$ descent, never enumerating substrings.

Algorithm

1. Build the SAM; compute $d[v]$ over states in decreasing len .
2. From the root, repeatedly scan edges in sorted order: locate the edge whose cumulative $1+d[u]$ contains k ; append its character; if k becomes 1 stop, else move into u with $k \leftarrow k-1$.
3. Output the accumulated string.

Dry run

Dry Run. $s = \text{"aba"}$; distinct substrings sorted: $\text{"a"}, \text{"ab"}, \text{"aba"}, \text{"b"}, \text{"ba"}$; find $k = 4$.

- From root, edge "a" covers $\text{"a"}, \text{"ab"}, \text{"aba"}$ ($1 + d = 3$ substrings). $k = 4 > 3$, subtract $\rightarrow k = 1$, move to edge "b" .
- Edge "b" : $k = 1 \Rightarrow$ answer is "b" .

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SAM {
5     struct State { int len, link; map<char,int> next; };
6     vector<State> st; int last;
7     SAM() { st.push_back({0, -1, {}}); last = 0; }
8     void extend(char c) {
9         int cur = st.size();
10        st.push_back({st[last].len + 1, -1, {}});
11        int p = last;
12        while (p != -1 && !st[p].next.count(c)) { st[p].next[c] = cur;
13            p = st[p].link; }
14        if (p == -1) st[cur].link = 0;
15        else {
16            int q = st[p].next[c];
17            if (st[p].len + 1 == st[q].len) st[cur].link = q;
18            else {
19                int clone = st.size();
20                st.push_back(st[q]);
21                st[clone].len = st[p].len + 1;
22                while (p != -1 && st[p].next[c] == q) { st[p].next[c] =
23                    clone; p = st[p].link; }
24                st[q].link = st[cur].link = clone;
25            }
26        }
27        last = cur;
28    }
29 };
30
31 int main() {
32     string s;
33     long long k;
34     cin >> s >> k;
35     SAM sam;
36     for (char c : s) sam.extend(c);
37
38     int m = sam.st.size();

```

```

37     vector<int> order(m);
38     iota(order.begin(), order.end(), 0);
39     sort(order.begin(), order.end(),
40         [&](int a, int b){ return sam.st[a].len > sam.st[b].len; });
41
42     vector<long long> d(m, 0);
43     for (int v : order)
44         for (auto& [c, u] : sam.st[v].next)
45             d[v] += 1 + d[u];                // distinct
46                                           continuations
47
48     string res;
49     int v = 0;
50     while (k > 0) {
51         for (auto& [c, u] : sam.st[v].next) { // sorted char
52             long long here = 1 + d[u];
53             if (k <= here) { res += c; k--; v = u; break; }
54             k -= here;
55         }
56     }
57     cout << res << "\n";
58     return 0;
}

```

Complexity Analysis

Time Complexity. $O(n \log \Sigma)$ to build and count; the descent is $O(|\text{answer}| \cdot \Sigma)$. **Space Complexity.** $O(n)$.

Key Takeaway

Ranking distinct substrings is a count-and-descend on the suffix automaton: $d[v] = \sum (1 + d[u])$ lets you skip whole branches. This “precompute subtree sizes, then walk to the k -th” pattern is the lexicographic-order workhorse, extended to multiplicity in the next section.

1.21 Substring Order II

Problem Statement. Given a string and an index k , find the k -th substring in lexicographic order *counting multiplicity*: a substring that occurs t times is counted t times, and equal occurrences are grouped together.

Pattern Recognition

This is Substring Order I weighted by occurrences. Each substring’s weight is its **endpos-set size** (how many times it occurs), so the per-state continuation count becomes an occurrence-weighted sum, and the descent subtracts weighted totals.

Weighting by occurrences

Let $\text{cnt}[u]$ be the endpos-set size of state u (its occurrence count, computed by propagating counts up the suffix-link tree). A path reaching u spells a substring occurring $\text{cnt}[u]$ times. Define the occurrence-weighted continuation total:

$$f[v] = \sum_{(c,u) \text{ edge from } v} (\text{cnt}[u] + f[u]),$$

where $cnt[u]$ counts the $cnt[u]$ occurrences of the substring ending exactly at u , and $f[u]$ counts all longer continuations with their multiplicities. The descent mirrors Order I: at an edge to u , if $k \leq cnt[u]$ the current prefix plus this character is the answer; else if $k \leq cnt[u] + f[u]$ move into u after subtracting $cnt[u]$; otherwise subtract the whole $cnt[u] + f[u]$ and continue.

Occurrences Replace the Unit Count

Order I used 1 per distinct substring; Order II uses $cnt[u]$, the number of occurrences. The identical descent then ranks substrings with repetition. Computing cnt is the same suffix-link propagation from Counting Patterns (Section 1.12).

Algorithm

1. Build the SAM; compute $cnt[u]$ (endpos sizes) via link-tree propagation.
2. Compute $f[v] = \sum_{(c,u)} (cnt[u] + f[u])$ in decreasing len order.
3. Descend from the root: at edge to u , if $k \leq cnt[u]$ output and stop; else if $k \leq cnt[u] + f[u]$ append, move to u , subtract $cnt[u]$; else subtract $cnt[u] + f[u]$.

Dry run

Dry Run. $s = \text{"aa"}$; substrings with multiplicity, sorted: “a”(pos 1), “a”(pos 2), “aa”; find $k = 2$.

- cnt of state “a” is 2; of “aa” is 1.
- Edge “a” to state “a”: $cnt = 2 \geq k = 2 \Rightarrow$ answer “a” (its second occurrence).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SAM {
5     struct State { int len, link; map<char,int> next; };
6     vector<State> st; vector<long long> cnt; int last;
7     SAM() { st.push_back({0, -1, {}}); cnt.push_back(0); last = 0; }
8     void extend(char c) {
9         int cur = st.size();
10        st.push_back({st[last].len + 1, -1, {}}); cnt.push_back(1);
11        int p = last;
12        while (p != -1 && !st[p].next.count(c)) { st[p].next[c] = cur;
13            p = st[p].link; }
14        if (p == -1) st[cur].link = 0;
15        else {
16            int q = st[p].next[c];
17            if (st[p].len + 1 == st[q].len) st[cur].link = q;
18            else {
19                int clone = st.size();
20                st.push_back(st[q]); cnt.push_back(0);
21                st[clone].len = st[p].len + 1;
22                while (p != -1 && st[p].next[c] == q) { st[p].next[c] =
23                    clone; p = st[p].link; }
24                st[q].link = st[cur].link = clone;
25            }
26        }
27    }
28 }

```

```

24     }
25     last = cur;
26 }
27 };
28
29 int main() {
30     string s;
31     long long k;
32     cin >> s >> k;
33     SAM sam;
34     for (char c : s) sam.extend(c);
35
36     int m = sam.st.size();
37     vector<int> order(m);
38     iota(order.begin(), order.end(), 0);
39     sort(order.begin(), order.end(),
40         [&](int a, int b){ return sam.st[a].len > sam.st[b].len; });
41     for (int v : order)
42         if (sam.st[v].link >= 0) sam.cnt[sam.st[v].link] += sam.cnt[v];
43
44     vector<long long> f(m, 0);
45     for (int v : order)
46         for (auto& [c, u] : sam.st[v].next)
47             f[v] += sam.cnt[u] + f[u];           // occurrence-
48                                                 // weighted
49
50     string res;
51     int v = 0;
52     while (k > 0) {
53         for (auto& [c, u] : sam.st[v].next) {
54             long long occ = sam.cnt[u];
55             if (k <= occ) { res += c; k = 0; break; }           // land
56             here
57             if (k <= occ + f[u]) { res += c; k -= occ; v = u; break; }
58             k -= occ + f[u];
59         }
60     }
61     cout << res << "\n";
62     return 0;
63 }

```

Complexity Analysis

Time Complexity. $O(n \log \Sigma)$ build and counts; descent $O(|\text{answer}| \cdot \Sigma)$. **Space Complexity.** $O(n)$.

Key Takeaway

Order II is Order I with the unit count replaced by each state's occurrence count. The two problems together show how one weighting choice on the same count-and-descend framework toggles between distinct and multiplicity ranking.

1.22 Substring Distribution

Problem Statement. Given a string of length n , for every length $L = 1, 2, \dots, n$ output the number of *distinct* substrings of that length.

Pattern Recognition

“Distinct substrings of each length” uses the same **suffix automaton** fact as distinct-substring counting, but resolved per length: each state contributes exactly one distinct substring to every length in its interval $(len[link], len]$, so a difference array over those intervals gives the per-length counts.

Each state covers a contiguous length range

A suffix automaton state v represents distinct substrings of every length from $len[link[v]] + 1$ up to $len[v]$, one distinct substring per length (they are the longer substring’s suffixes, all sharing v ’s endpos class). Therefore the number of distinct substrings of length L equals the number of states whose interval contains L . Add $+1$ over each state’s interval with a difference array, then take a prefix sum to read off the count for every length.

Difference Array Over State Intervals

Instead of iterating each length inside each state ($O(n^2)$), mark $diff[len[link] + 1] += 1$ and $diff[len + 1] -= 1$ per state, then prefix-sum once. Every state’s whole length interval is applied in $O(1)$.

Algorithm

1. Build the suffix automaton.
2. For each non-root state v : $diff[len[link[v]] + 1] += 1$; $diff[len[v] + 1] -= 1$.
3. Prefix-sum $diff$; output entries for $L = 1 \dots n$.

Dry run

Dry Run. $s = \text{“aba”}$.

- Distinct substrings by length: length 1: “a”, “b” $\rightarrow 2$; length 2: “ab”, “ba” $\rightarrow 2$; length 3: “aba” $\rightarrow 1$.
- The interval contributions sum, via the difference array, to 2, 2, 1.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SAM {
5     struct State { int len, link; map<char, int> next; };
6     vector<State> st; int last;
7     SAM() { st.push_back({0, -1, {}}); last = 0; }
8     void extend(char c) {
9         int cur = st.size();
10        st.push_back({st[last].len + 1, -1, {}});
11        int p = last;
12        while (p != -1 && !st[p].next.count(c)) { st[p].next[c] = cur;
13            p = st[p].link; }
14        if (p == -1) st[cur].link = 0;
15        else {

```

```

15         int q = st[p].next[c];
16         if (st[p].len + 1 == st[q].len) st[cur].link = q;
17         else {
18             int clone = st.size();
19             st.push_back(st[q]);
20             st[clone].len = st[p].len + 1;
21             while (p != -1 && st[p].next[c] == q) { st[p].next[c] =
22                 clone; p = st[p].link; }
23             st[q].link = st[cur].link = clone;
24         }
25         last = cur;
26     }
27 };
28
29 int main() {
30     string s;
31     cin >> s;
32     int n = s.size();
33     SAM sam;
34     for (char c : s) sam.extend(c);
35
36     vector<long long> diff(n + 2, 0);
37     for (int v = 1; v < (int)sam.st.size(); v++) {
38         int lo = sam.st[sam.st[v].link].len + 1;    // shortest length
39         // in v
40         int hi = sam.st[v].len;                    // longest length
41         // in v
42         diff[lo] += 1;
43         diff[hi + 1] -= 1;
44     }
45     long long run = 0;
46     for (int L = 1; L <= n; L++) { run += diff[L]; cout << run << " ";
47 }

```

Complexity Analysis

Time Complexity. $O(n \log \Sigma)$ to build; $O(n)$ for the difference array and output. **Space Complexity.** $O(n)$.

Key Takeaway

Each suffix-automaton state owns a contiguous interval of substring lengths; a difference array turns “count distinct substrings per length” into one linear sweep. This closes the chapter’s suffix-automaton arc: presence, count, position, distinct count, ranking, and now the full length distribution – all from one structure.